

Rowan Software Engineering Project Ideas

<https://gemini.google.com/app/ff573c21b72d5ecf>

User prompt: what would be a good project to do for our software engineering class at Rowan its a group of 6 students, some similar backgrounds, theyre all solid at java, ive done some spring boot and python, theyre solid at python, one of them is good at sql, and wee need something that we can use a scrum development process, and that is relatively difficult and worth doing, we'll be using claude and other ai tools, and we'll be evaulated on our scrum reports and github management/branches comits etc. i guess it should be something that is meaningful and that we can showcase idk if we'll do ports to the web or having our server set up with spring boot as it is messy to set up but idk, what do you think

Response: For a group of six with strong skills in Java, Python, and SQL—and looking to maximize evaluation criteria like Scrum management, GitHub branching, and code metrics—you want a **distributed or multi-service architecture**.

Avoid spending weeks configuring complex server deployments or web hosting setups if that feels like unnecessary overhead. Instead, build a **hybrid or CLI-driven distributed system** or leverage containerized microservices running locally via Docker Compose.

1. Developer Tooling: Local Code Intelligence & Repository Health CLI/Dashboard

Build a developer tool that analyzes GitHub repositories, tracks code quality metrics, runs custom static analysis, and provides AI-driven architectural advice or test case generation.

- **Architecture:**

- **Core Engine (Java / Spring Boot or CLI):** Parses source files, generates abstract syntax trees (ASTs), tracks cyclomatic complexity, and computes change frequency across commits.
- **AI & Analytics Worker (Python):** Handles vector embeddings, interfaces with LLM APIs (Claude API), and runs machine learning / NLP tasks like generating docstrings or detecting duplicated logic patterns.
- **Database (PostgreSQL / SQL):** Stores historical repo metrics, contributor activity, test coverage runs, and snapshot evaluations.
- **Interface Options:** Either a modern CLI (terminal UI like Textual in Python or Rich in Java) or a minimal desktop GUI (JavaFX / Electron) to avoid full web server setup hassle.

- **Why it fits your team:**

- **Scrum & GitHub Division:** Clear boundaries. Two engineers on Java static analysis, two on Python AI integration/pipeline, one on PostgreSQL schema optimization/indexing, and one on Git/GitHub API integrations and TUI display.
- **Scrum Artifacts:** Generates clean user stories ("As a developer, I want to scan my branch before opening a PR to check test coverage degradation").
- **Showcase Value:** You can use your own tool to analyze your team's project repository during final presentations.

2. Multi-Agent AI Task Coordinator & Queue Engine

Build a distributed orchestration system where users queue complex background tasks (e.g., automated web scraping, data transformation, multi-step LLM workflows), and distributed worker nodes process them asynchronously.

- **Architecture:**

- **Backend & API (Java Spring Boot):** Handles task creation, JWT authentication, state management, and worker heartbeats.
- **Processing Workers (Python):** Independent worker scripts running locally or in Docker containers that pick up queued jobs, run AI workflows or scraping jobs, and stream results back.
- **Database (PostgreSQL / SQL):** Complex relational schema managing job queues, priority scheduling, retry logs, execution times, and payload history.
- **Message Broker:** Redis or RabbitMQ for job distribution.
- **Why it fits your team:**
 - **High Technical Depth:** Involves concurrency, locking mechanisms, database transactions, retry strategies, and asynchronous job states.
 - **Spring Boot Efficiency:** Spring Boot excels at building REST APIs and task scheduling engines without needing complex frontend hosting—you can test everything via Postman/OpenAPI or a lightweight dashboard.
 - **Clear GitHub Branching:** Feature branches split cleanly between `feature/spring-scheduler`, `feature/sql-queue-optimizations`, `feature/python-worker-engine`, and `feature/claude-agent-executor`.

3. Automated Academic / Campus Asset Scheduling & Intelligent Optimization Engine

Build a smart resource and room booking platform tailored for campus labs, study spaces, or hardware checkout.

- **Architecture:**
 - **Core Application (Java / Spring Boot REST API):** Encapsulates business logic, conflict resolution, authorization roles (student, faculty, admin), and notification hooks.
 - **Optimization & AI Module (Python):** Runs constraint satisfaction algorithms (e.g., OR-Tools) or AI-driven scheduling to auto-assign lab equipment and room slots based on requested criteria, group size, and priority.
 - **Database Layer (SQL / PostgreSQL):** Requires heavy, well-structured relational schema design (complex joins, temporal constraints, recursive queries for recurring bookings).
- **Why it fits your team:**
 - **Scrum Friendly:** Easy to slice into sprints: Sprint 1 (Basic Schema & Auth), Sprint 2 (Booking Engine & CRUD), Sprint 3 (Python Optimization Engine), Sprint 4 (AI Conflict Resolution & Analytics).
 - **Relatively Difficult:** Handling edge cases—time-zone overlaps, recurring schedule conflicts, automated queue re-ordering when bookings are canceled—provides genuine algorithmic challenge.

How to Excel on Evaluation Metrics

- **Leverage Claude & AI Tools Strategically:** Use AI to draft unit tests, write boilerplate DTOs, and generate comprehensive documentation. Highlight in your Scrum reports how AI was integrated into your sprint velocity.
- **Master Git/GitHub Workflow:** Strictly enforce a `main -> develop -> feature/ticket-number` branching strategy. Require PR reviews from at least two teammates before merging.

- **Scrum Structure:** Rotate the Scrum Master role every 2 sprints so every team member gets project management artifacts on their record. Use GitHub Projects or Jira for burndown charts, backlog grooming, and sprint retrospectives.

If you want to dive deeper into any of these three ideas, let me know which direction appeals most to the group!

Check out this quick overview of building scalable RESTful APIs with Java Spring Boot and PostgreSQL: [Spring Boot & PostgreSQL REST API Guide](#) — This video walks through clean architectural layering, entity relationships, and database persistence in Spring Boot, which aligns directly with setting up your group's core backend logic smoothly.

http://googleusercontent.com/youtube_content/1

Sources

1. [Course number and name: CS 04321: Software Engineering I](#)

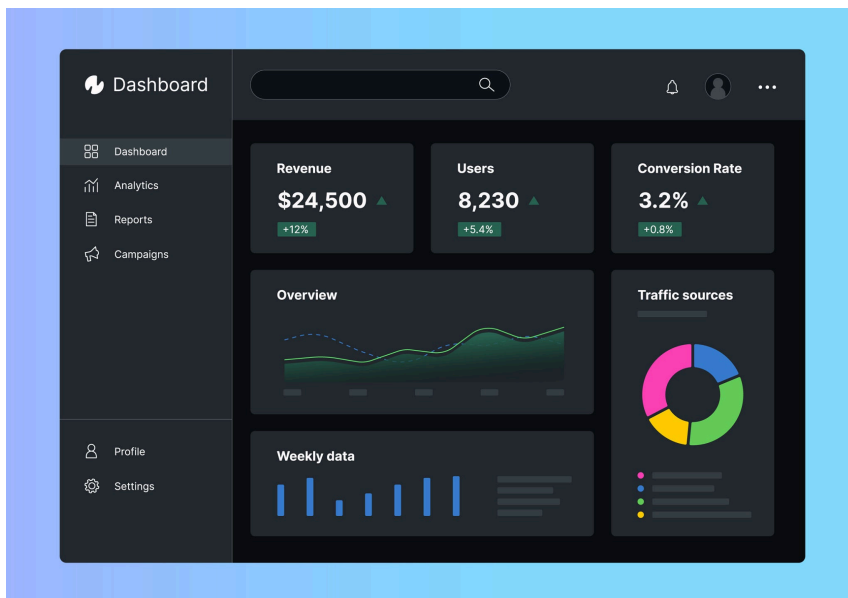
User prompt: maybe something more visual, more ui, maybe some javascript and html css?

Response: Adding a rich frontend—using HTML/CSS, JavaScript (or React/Vue), and canvas/charting libraries—makes a project much more engaging to present during final demos while letting team members split cleanly between UI/UX and heavy backend systems.

Here are three UI-heavy, visually impressive web application projects tailored for a group of 6:

1. Visual Repository Code Graph & Architectural Health Dashboard

An interactive Web app that turns any software codebase into a dynamic, 3D/2D node-based network visualizer (using **three.js** or **D3.js**), showing how classes, packages, and microservices depend on each other, highlighting code smells, and rendering real-time AI architectural recommendations.



- **Frontend (HTML/CSS, JS / React, D3.js or Three.js):**
 - Interactive canvas where file nodes scale based on lines of code and color-code based on test coverage or cyclomatic complexity.
 - Side drawer for live Claude/AI code review, refactoring suggestions, and visual diffs.

- **Backend (Java / Spring Boot):**

- Parses Java/Python source code, generates dependency trees, and serves AST graph structures via REST APIs and WebSockets.

- **AI & Analytics (Python):**

- Interfaces with Claude API to analyze complex dependency cycles or security risks and send structured JSON payloads to the UI.

- **Database (PostgreSQL / SQL):**

- Stores past scan results, user project configurations, and commit histories.

2. Real-Time Collaborative System Architecture Sandbox & Simulator

A browser-based drag-and-drop system design canvas (like Figma or Draw.io) where users can build cloud architecture diagrams (databases, load balancers, API gateways), push "Start Simulation", and watch real-time animated traffic packets flow through their system with simulated latency and server failures.

- **Frontend (HTML5 Canvas / SVG, JavaScript, Tailwind CSS):**

- Drag-and-drop canvas with live connection lines, node status animations, and metrics panels for throughput and error rates.
- Real-time multi-user cursor tracking using WebSockets.

- **Backend Engine (Java / Spring Boot):**

- Runs the discrete-event simulation core: calculates queue bottlenecks, simulates server dropouts, and streams packet updates to the frontend at 60 FPS via WebSockets.

- **AI Assistant (Python):**

- "AI Architecture Critic": evaluates the canvas state and alerts users on visual single-points-of-failure or bottlenecked SQL databases.

- **Database (SQL):**

- Stores saved diagrams, component templates, user accounts, and test scenarios.

3. Intelligent Campus/Event Logistics & Dynamic Map Operations Center

A live "Mission Control" map dashboard designed for managing large physical campus facilities, smart parking tracking, or event resource scheduling with interactive geospatial map layers (using **Mapbox GL JS** or **Leaflet**).

- **Frontend (JS, HTML/CSS, Mapbox GL JS, Chart.js):**

- Rich dark-mode dashboard displaying interactive 2D/3D map floor plans, heatmaps of room usage, live sensor feeds, and filterable event logs.
- Interactive scheduling timeline UI with drag-and-drop booking blocks.

- **Backend Services (Java Spring Boot REST & WebSocket Server):**

- Handles authentication, booking rules, time-conflict checks, and pushes live updates to the frontend maps whenever space is reserved or updated.

- **Optimizer & AI (Python):**

- Solves spatial-temporal optimization problems (e.g., automatically placing hardware assets or re-routing room assignments based on attendance) and runs Claude AI to answer queries like *"Which rooms are underutilized on Tuesdays?"*
- **Database (PostgreSQL with PostGIS):**
 - Leverages SQL spatial queries (ST_Contains, ST_Distance) to store geo-coordinates, room polygons, and time-stamped booking logs.

How to Structure a 6-Person Web Project in Scrum

Role	Responsibility	Primary Tech Stack
Frontend Lead 1	UI Components, Layout, CSS Styling	HTML5, CSS/Tailwind, JS Framework
Frontend Lead 2	Canvas / Visuals / Graphics Pipeline	D3.js, Three.js, or Mapbox
Backend Core 1	REST API, Auth, Spring Controllers	Java, Spring Boot
Backend Core 2	Real-time Sockets, System State	Java / WebSockets
Database & SQL Specialist	Schema, Complex Queries, Persistence	PostgreSQL / SQL
AI & Pipeline Specialist	Claude API Integration, Data Processing	Python, FastAPI / Flask

User prompt: option 1 sounds interesting, building an interactive analytics visual dashboard is interesting bc it combines all languages but analytics of what and what kind of data will it need to function and how difficult would it be to set up to a server, etc

Response: http://googleusercontent.com/map_content/0

Building a visual code analytics dashboard (**Option 1**) is one of the best choices for a 6-person group project because it solves a real developer problem, has clear architectural separation, and can be easily demonstrated right on your laptop during final presentations without any expensive cloud server hosting.

Below is a complete breakdown of **what data to analyze**, **how to get that data**, **how hard server deployment is**, and **how to structure the visual dashboard**.

1. What Data Will It Analyze? (The Core Domain)

Instead of vague web traffic, you analyze **Git Repositories and Source Code** (including your own project repository as it evolves during the semester).

Key Datasets & Metrics to Collect:

- **Source Code Structure & Dependency Graphs:**
 - Parse .java and .py files to identify classes, methods, and functions.
 - Map import statements and cross-file calls to build a **Dependency Matrix** (which classes depend on which).

- **Git History & Commit Activity:**

- Query local Git logs via CLI or GitHub API to extract: line insertions/deletions, author velocity, commit frequency, and "hotspots" (files edited frequently across sprints).

- **Code Quality & Complexity Metrics:**

- **Cyclomatic Complexity:** Measuring conditional statements (if, for, while) to flag hard-to-maintain code blocks.
- **Code Churn & Bus Factor:** Identifying which files are heavily touched by only one team member.

- **AI-Generated Code Health & Debt Insights:**

- Feed file diffs or complex functions to the **Claude API** to generate structured JSON feedback: code smell severity ratings, architectural vulnerability warnings, and automated unit test suggestions.

2. How the Dashboard Visualizes the Data

The frontend is where the visual element shines using HTML5/CSS, JavaScript, and visualization libraries:

- **Interactive Dependency Graph (using D3.js or Force Graph JS):**

- Nodes represent files or classes; edges represent dependencies/imports.
- Node sizes scale dynamically based on lines of code (LOC) or commit count.
- Nodes color-code automatically: **Green** (low complexity / high test coverage), **Yellow** (medium debt), **Red** (high complexity / severe code smell flagged by Claude).

- **Git Hotspot Heatmap & Velocity Charts (using Chart.js or Recharts):**

- Time-series line charts tracking team commit frequency across Scrum sprints.
- Heatmaps showing which modules had the highest lines changed during a specific sprint.

- **Side-by-Side AI Refactoring Drawer:**

- Clicking any node opens a slide-over panel displaying file stats, flagged code smells, and Claude's suggested refactored code block with visual diff highlighting.

3. Server Setup & Hosting: Is It Difficult?

Short answer: No, because you don't need a public cloud server.

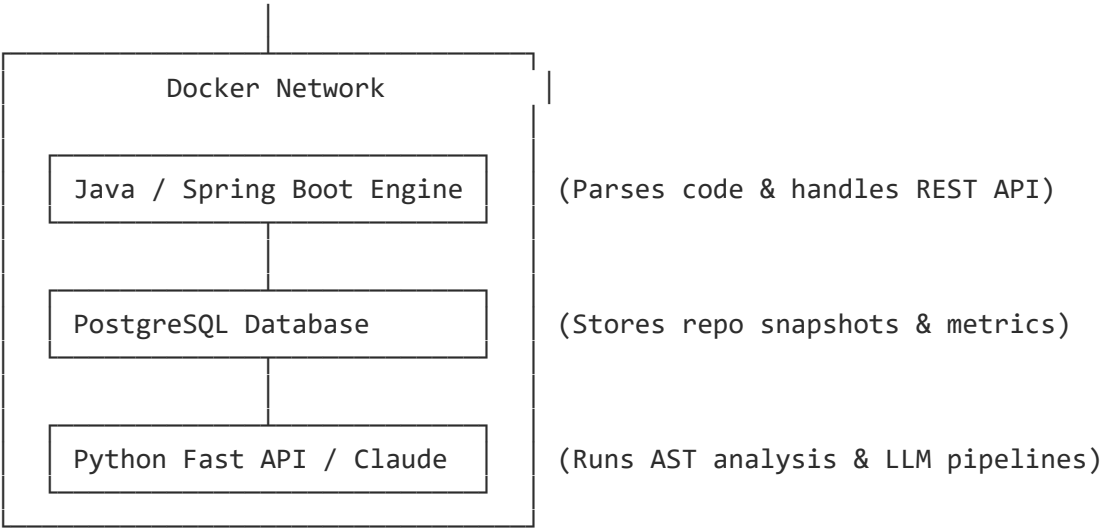
[RichListCard: 15]

For a class at Rowan University, you do **not** need to spend money on AWS/GCP or wrestle with DNS, SSL certificates, and web hosting config to get a high grade. Evaluation in software engineering courses focuses on **architecture, Scrum execution, unit test coverage, and Git history**, not production cloud infrastructure.

The Best Setup: Containerized Local Environment (Docker Compose)

Your entire application can run locally on any team member's laptop using **Docker Compose**:

```
[ Web Browser Frontend ]
  (HTML/CSS/JS/D3)
      |
  HTTP / WebSockets
```



Why Docker Compose makes setup easy:

- 1. **Zero Deployment Overhead:** You write a single `docker-compose.yml` file. Anyone on the team types `docker compose up` and the whole stack (Spring Boot, Python service, PostgreSQL, and static web files) spins up in seconds.
- 2. **Consistent Environment:** Eliminates the "it works on my machine" problem across Windows and macOS.
- 3. **Showcase Ready:** During your final demo, you run it locally on `localhost:8080`, point it at a open-source GitHub repo (or your own group's repo), and watch the graph render live.

4. Technical Workload Distribution for a 6-Person Group

Teammate	Focus Area	Responsibilities	Primary Tech
Developer 1 (Java / Spring)	Core API & Git Integration	Interfacing with Git API / <code>jgit</code> , parsing file trees, serving REST endpoints for graph data.	Java, Spring Boot
Developer 2 (Java / Spring)	Metric Calculation Engine	Computing cyclomatic complexity, code churn rates, and handling WebSocket feeds.	Java, Spring Boot
Developer 3 (Python)	Static Analysis & Claude Pipeline	Executing AST parsing in Python, formatting prompts, calling Claude API for code reviews.	Python, FastAPI / Flask
Developer 4 (SQL Specialist)	Schema & Historical Database	Designing PostgreSQL tables for scans, commits, file nodes, edges, and indexing for fast queries.	SQL, PostgreSQL, Liquibase
Developer 5 (JS / Frontend)	Graph Visualization Pipeline	Building interactive network graphs using D3.js or Force-Graph, handling node drag/zoom/click events.	JavaScript, HTML5 Canvas, D3.js
Developer 6 (JS / CSS / UI)	Dashboard Layout & Analytics	Crafting responsive layout, sprint velocity charts (Chart.js), drawer component, and dark-mode styling.	HTML/CSS, JS, Tailwind

Why Evaluators Love This Project

- **Demonstrates Engineering Maturity:** Building developer tools demonstrates a deep understanding of software design patterns and static analysis.
 - **Natural Scrum Sprints:**
 - **Sprint 1:** Set up Docker Compose, basic SQL schema, and Spring Boot REST skeleton.
 - **Sprint 2:** Java file parsing + basic D3.js node rendering on frontend.
 - **Sprint 3:** Git commit history tracking + Python Claude API integration.
 - **Sprint 4:** Dashboard UI polish, AI code review drawer, and performance tuning.
 - **Self-Referential Demo:** During final presentations, you can feed **your group's own GitHub repository** into your tool and show the historical visual progress of your team's commits over the semester.
-

User prompt: ok i like this, but not about code or github, i would like to make this for anything more meaningful other things like finance or something that can actually be functional i dont think anyone cares about analyzing code,

Response: Redirecting this visual architecture toward **personal finance and automated wealth building** gives you a tool that is genuinely useful, highly practical, and visually impressive.

Rather than basic expense trackers (which just show pie charts), you can build a **Visual Financial Cascade & Automated Wealth Engine**. This treats a user's entire financial life as an interactive flow diagram: income enters at the top, streams through dynamic rules (taxes, high-yield savings, 401k match, debt payoff strategies, investment portfolios), and branches out visually.

1. The Core Concept: What Does It Do?

Think of it as a **SimCity for your money**. Instead of static bank statements, users build a visual blueprint of their monthly cash flow and net worth growth over time.

- **Interactive Cash-Flow Pipeline:**
 - Nodes represent income sources (salary, side hustles), accounts (checking, Roth IRA, brokerage), and goals (emergency fund, house down payment, debt payoff).
 - Animated visual pipes show money flowing through the system each month based on custom rules (e.g., *"If checking balance > \$3,000, route 60% to Roth IRA and 40% to high-yield savings"*).
- **Interactive Scenario & Monte Carlo Simulator:**
 - Users can adjust sliders (e.g., inflation rate, market return, salary raises, job loss simulation) and instantly see their 5, 10, and 30-year net worth trajectories rendered on dynamic charts.
- **AI Financial Strategist (Claude Integration):**
 - The user can ask: *"How much faster do I hit my home down payment goal if I switch from paying minimum debt to the Avalanche Method?"*
 - Claude reads the graph structure and financial metrics, then returns structured strategy recommendations and updates the visual graph live.

2. What Data Does It Use?

To make it fully functional without needing live, paywalled bank connections (like Plaid API, which requires paid developer keys):

- **Structured Financial Profiles (JSON/SQL):** Income tiers, fixed expenses (rent, utilities), variable expenses, asset allocation ratios, debt interest rates (APR), and tax brackets.
- **Historical Market & Economic Data:** Free financial APIs (like Yahoo Finance API or Alpha Vantage) or pre-loaded CSV datasets to run historical index simulations (S&P 500 historical returns, historical inflation rates).
- **Goal Milestones:** Target dates, compounding frequency, emergency fund minimum thresholds.

3. How the Dashboard Visualizes the Data

- **Sankey Diagram / Directed Node Canvas (D3.js / Chart.js):**
 - Displays the exact path every dollar takes from paycheck to investments.
 - Node thickness scales proportionally to the dollar amount.
 - Color-coded flows: **Green** (Wealth Building / Savings), **Blue** (Essential Expenses), **Red** (High-Interest Debt / Losses).
- **Multi-Scenario Net Worth Projection Graph:**
 - Interactive line chart displaying baseline trajectory vs. optimized strategy side-by-side.
- **Debt Avalanche vs. Snowball Interactive Visualizer:**
 - An interactive payoff simulator showing month-by-month debt breakdown and total interest paid under different strategies.

4. Technical Workload Distribution for a 6-Person Group

This domain maps perfectly to your group's skill set, giving everyone a distinct component to own for Scrum evaluation:

Teammate	Primary Role	Domain Responsibility	Tech Stack
Developer 1 (Java / Spring)	Core Financial Engine	Debt payoff algorithms (Avalanche vs. Snowball), compound interest math, tax-bracket logic.	Java, Spring Boot
Developer 2 (Java / Spring)	REST API & State	Managing scenario states, rule evaluation pipeline, serving simulation endpoints.	Java, Spring Boot
Developer 3 (Python)	Monte Carlo & AI Advisor	Running market return probability simulations (NumPy/Pandas) & prompt pipeline with Claude.	Python, FastAPI / Flask
Developer 4 (SQL Specialist)	Financial Schema	Complex SQL queries for historical ledger entries, portfolio histories, compound yield snapshots.	SQL, PostgreSQL
Developer 5 (JS / Frontend)	Cash Flow Canvas	Building the node graph / Sankey cash-flow visualizer and node interactions.	JavaScript, D3.js

Teammate	Primary Role	Domain Responsibility	Tech Stack
Developer 6 (JS / CSS / UI)	Dashboard Layout	Responsive financial dashboard, slider controls, scenario comparison charts, and dark-mode CSS.	HTML5, CSS/Tailwind, Chart.js

5. Why This Project Excel in a Scrum Class

1. Clearly Defined Sprints:
- **Sprint 1:** PostgreSQL schema design, basic Spring Boot REST API for account CRUD, and local Docker Compose setup.

◦ **Sprint 2:** Java calculation engine (amortization tables, compound interest) + basic D3.js cash flow graph.

◦ **Sprint 3:** Python Monte Carlo simulation engine + Claude AI financial advisor endpoint.

◦ **Sprint 4:** Scenario slider UI controls, side-by-side comparison charts, and final UI polishing.
2. Easy Docker Local Setup: The entire stack runs on localhost via Docker Compose.
3. High Portfolio & Demo Value: When presenting to your class, you can input a real-world scenario—like a college graduate starting a \$75,000 job with \$20,000 in student loans—and visually show how your tool optimizes their 10-year path to wealth.

User prompt: sounds good but i dont understand where the data comes from again, i wouldnt want to hardcode it, is there any way around it so we can make it look realistic

Response: You definitely shouldn't hardcode static numbers. To make the project feel realistic, dynamic, and fully functional, the data should come from **a mix of free live APIs, realistic seed database profiles, and dynamic user input.**

Here is where the data will actually come from and how it flows through your system:

1. Live Market Data & Historical Economic APIs (Python Service)

Your Python backend can fetch real-time and historical financial data using free, open-source libraries and APIs —no credit card or paid developer subscription required.

- **yfinance (Python Library):** Pulls live stock, ETF (e.g., S&P 500 / VOO), dividend yields, and historical return series directly from Yahoo Finance with zero API keys.

• **Alpha Vantage / Finnhub (Free Tier APIs):** Provides macroeconomic indicators like current inflation rates (CPI), federal interest rate changes, and sector performance metrics.

• **How it's used:** When a user models a 401(k) portfolio (e.g., 80% S&P 500, 20% Bonds), your Python service queries yfinance to fetch the actual 10-year compound annual growth rate (CAGR) and volatility for those assets, running the projection on real market data.

2. The Database Seed Generator (SQL Specialist)

Since real users won't connect their actual bank logins during a class demo, your team's **SQL specialist** writes seed scripts (using tools like Faker in Python or SQL scripts) that populate PostgreSQL with **realistic archetype profiles.**

For your presentation, you can load pre-configured profiles at the click of a button:

- **Archetype A: "Recent College Grad"** (\$68k salary, \$28k student loans at 6.5% APR, \$2.5k checking balance, 401k with 4% company match).
- **Archetype B: "Mid-Career Family"** (\$140k household income, \$320k mortgage at 6.1% APR, \$15k emergency fund, Roth IRA).
- **Archetype C: "Aggressive FIRE Strategy"** (Financial Independence, Retire Early—\$110k income, 50% savings rate, heavy index fund allocation).

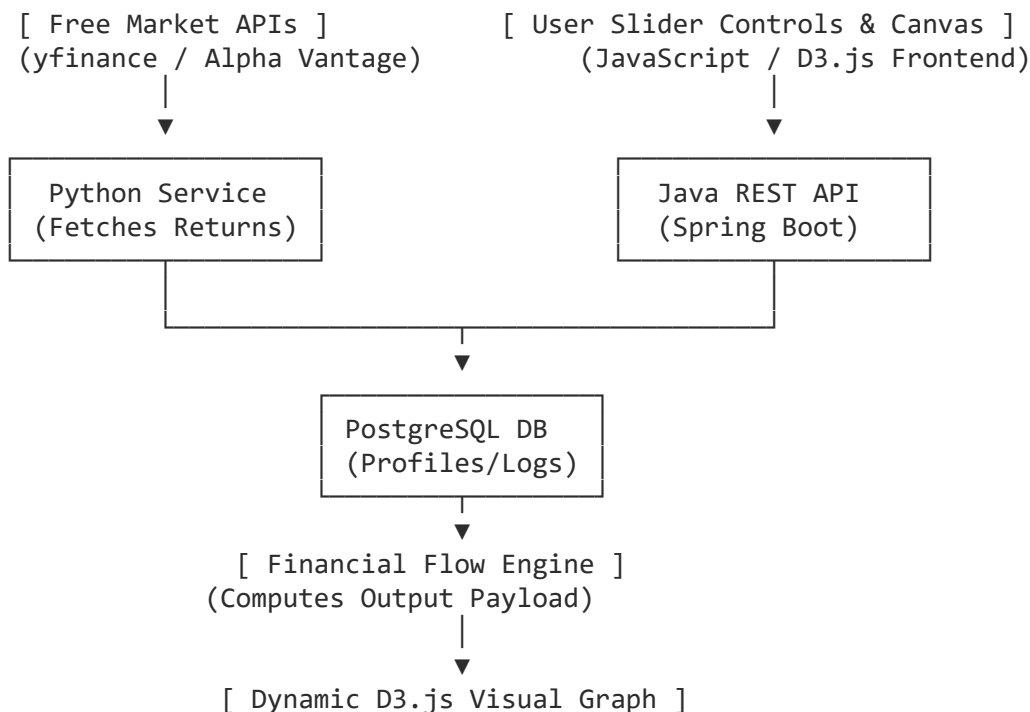
This allows evaluators to test the system with completely different scenarios in seconds.

3. Interactive UI Controls (Frontend JavaScript)

On the frontend, user inputs act as real-time variables that instantly feed the backend math engine:

- Sliders for **Income, Savings Rate, Monthly Expenses, Interest Rates, and Retirement Age**.
- Checkboxes for **Debt Strategies** (Avalanche vs. Snowball method).
- Drag-and-Drop Nodes on the canvas to add new streams (e.g., adding a "Side Freelance Income: \$800/mo" node or a "High-Yield Savings: 4.5% APY" node).

4. How the Data Pipeline Works Together (Architecture)



1. **User Request:** The user selects the "Recent College Grad" profile and drags a slider to increase their 401(k) contribution from 3% to 8%.
2. **Spring Boot (Java):** Intercepts the request, pulls the user's stored ledger items (rent, student loans) from PostgreSQL, and routes a payload to Python.
3. **Python & Claude:** Python pulls the 10-year S&P 500 average via yfinance, computes a Monte Carlo probability distribution for their portfolio, and prompts Claude for strategy recommendations.

4. **JSON Response:** The backend returns a structured JSON payload containing month-by-month cash balances, debt payoff dates, and node connections.
5. **D3.js Frontend:** The interactive diagram instantly re-renders: the "401(k)" node pipe gets thicker, the "Student Loan" debt node dissolves 24 months earlier, and the net worth graph curves upward.

This setup ensures zero hardcoded values, real market mechanics, and a presentation demo that feels like a finished, professional SaaS product.

Sources

1. [How to Get Yahoo Finance Data in Python \(2026\) - api - Scrapfly](#)

User prompt: and then how would a user or customer use it, remember that we have to do user stories and stuff like that, we have to pretend like we have a client

Response: To make your Scrum reports, product backlog, and user stories look authentic for your professor and client evaluations, you structure the application around **three distinct Client Personas (User Roles)**:

1. **The Retail Client (End-User / Individual Saver):** A person using the app to manage their personal cash flow, model retirement goals, and automate debt payoff.
2. **The Certified Financial Planner / Advisor (B2B Client):** A professional who onboard clients, creates custom scenario blueprints, and exports interactive strategy reports.
3. **The System Administrator:** An admin managing market API connections, archetype templates, and system health.

1. How a Customer Uses the Product (The End-to-End User Flow)

[1. Onboarding] [2. Interactive Canvas] [3. AI Strategy & Simulation]
 Select Archetype or → Drag & Connect Financial → Run Monte Carlo & Ask Claude
 Input Income/Expenses Nodes (Sankey Diagram) for Optimization Advice

1. **Onboarding & Persona Selection:** The user logs in and either selects a pre-configured template (e.g., *"Recent College Grad with Student Loans"*) or connects custom data points (Income, Rent, Credit Card Debt, 401k Match).
2. **Canvas Mapping:** The user sees their money flowing in real time through an interactive **Sankey node graph**. They can drag a new node (e.g., *"Side Hustle: +\$500/mo"*) and connect it directly to *"High-Yield Savings"* or *"Student Loan Principal"*.
3. **Running Simulations & AI Advice:** The user clicks **"Simulate 5-Year Net Worth"**. The system executes Monte Carlo simulations using live market yields (yfinance), recalculates the debt payoff timeline, and presents an interactive side-by-side comparison. Claude provides actionable tips on how to save on total interest.

2. Ready-to-Use Epic & User Stories (for your GitHub / Jira Backlog)

In Scrum, features are organized into **Epics** and broken down into **User Stories** following the standard template: As a [role], I want to [action], so that [business value].

Epic 1: Financial Graph Visualizer & Canvas

- **User Story 1.1 (Frontend Lead):**

- *Story:* As an **End-User**, I want to drag and drop financial nodes (income, debt, investments) onto an interactive canvas, so that I can visually map out my monthly cash flow.
- *Acceptance Criteria:* Canvas updates node link thickness dynamically based on dollar amount; supports adding/deleting nodes.
- **User Story 1.2 (Java Backend):**
 - *Story:* As an **End-User**, I want the backend to automatically calculate tax bracket withholdings and 401(k) company matches when I edit my salary node, so that my net cash flow calculations are accurate.
 - *Acceptance Criteria:* Spring Boot REST API recalculates net income and returns updated link flows in under 200ms.

Epic 2: Market Data & Monte Carlo Engine

- **User Story 2.1 (Python / AI Lead):**
 - *Story:* As a **Financial Advisor**, I want to run 1,000 Monte Carlo market simulations on a client's portfolio using real historical S&P 500 returns, so that I can show them their probability of hitting retirement goals under different market conditions.
 - *Acceptance Criteria:* Python service pulls live tickers via yfinance and outputs 10th, 50th, and 90th percentile net worth curves.
- **User Story 2.2 (SQL Specialist):**
 - *Story:* As a **System Admin**, I want to persist historical simulation runs and ledger states in PostgreSQL, so that users can load and compare past financial plans.
 - *Acceptance Criteria:* Schema includes relational tables for users, financial_nodes, edges, and simulation_snapshots with foreign key integrity.

Epic 3: AI Financial Strategist (Claude Integration)

- **User Story 3.1 (Python Lead / Full-Stack):**
 - *Story:* As an **End-User**, I want to click an "Optimize Strategy" button to ask Claude how to pay off my debt faster, so that I can get personalized, actionable steps without hiring an expensive advisor.
 - *Acceptance Criteria:* System sends the current graph state JSON to the Claude API and renders suggested node adjustments alongside a visual diff.

3. How to Structure Sprint Deliverables for Scrum Evaluation

When your professor inspects your GitHub repository and Sprint reports, structure your workflow as follows:

Scrum Artifact	How Your Team Implements It on GitHub
GitHub Projects / Board	Columns for <i>Backlog</i> , <i>Sprint Backlog</i> , <i>In Progress</i> , <i>In Review (PR)</i> , and <i>Done</i> . Every issue links to a User Story ID (e.g., US-1.1).
Git Branching Strategy	Strictly enforce feature branches named after story IDs: feature/US-1.1-sankey-canvas merging into develop via Pull Request with at least 2 peer code reviews .

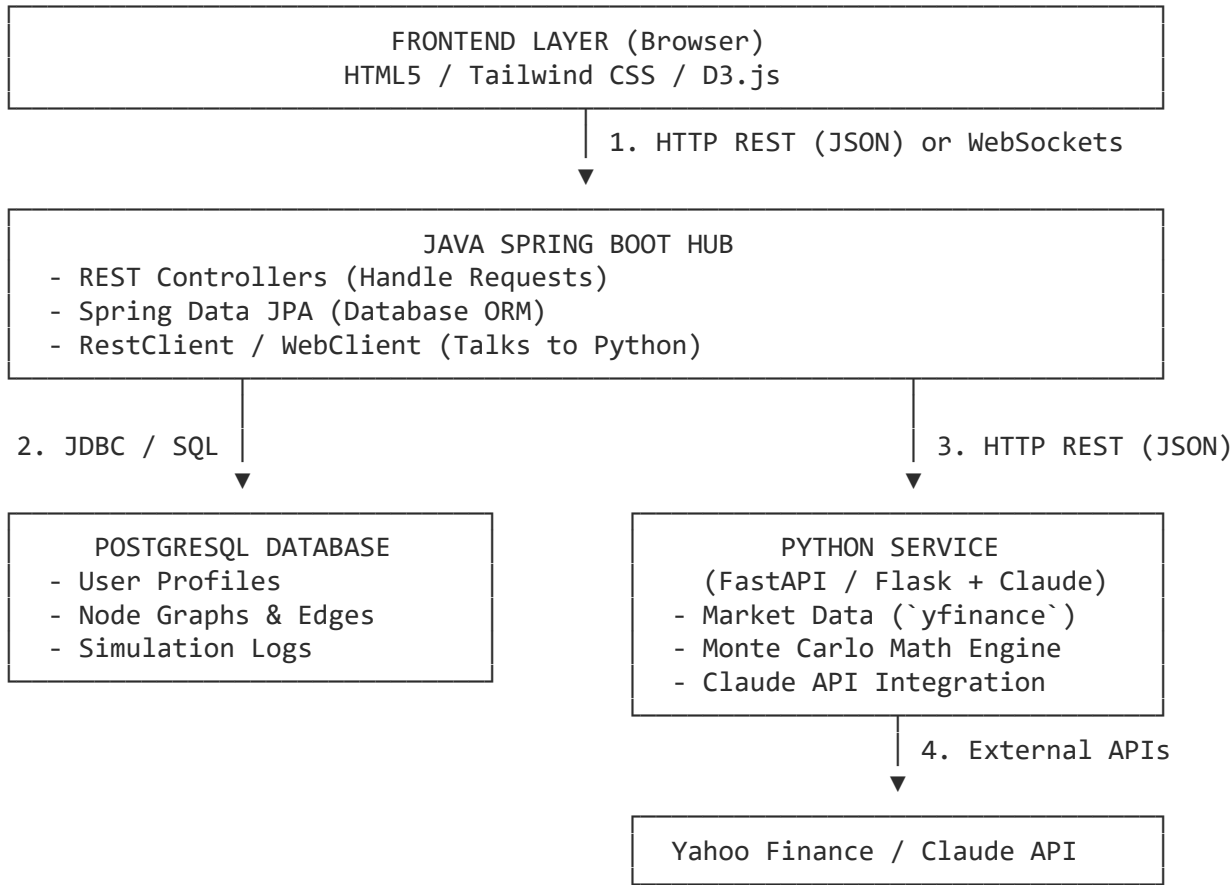
Scrum Artifact	How Your Team Implements It on GitHub
Sprint Retrospectives	At the end of each 2-week sprint, submit a brief retro log summarizing: <i>What went well</i> , <i>What was blocked</i> , and <i>Velocity metrics</i> (Story points completed vs. planned).

User prompt: how would we connect all of those together, the python portion, sql portion, html css js, to the java springboot

Response: Connecting these components together relies on a clear, layered **REST & WebSocket architecture** running in a single Docker network on your laptop.

Spring Boot acts as the **central hub** (orchestrator). The frontend talks only to Spring Boot, Spring Boot queries PostgreSQL and delegates complex data/AI jobs to Python, and Python calls market APIs and Claude.

The Communication Flow



Step-by-Step Connection Protocols

1. HTML / CSS / JS → Java Spring Boot

- **Protocol:** HTTP REST APIs (JSON) or WebSockets for real-time updates.
- **How it works:** Your frontend JavaScript sends standard `fetch()` or `axios` requests to Spring Boot controllers listening on `localhost:8080/api/...`

- **Example JS Call:**

```
// Fetch the user's financial graph node structure from Spring Boot
async function loadFinancialGraph(userId) {
  const response = await fetch(`http://localhost:8080/api/graph/${userId}`);
  const graphData = await response.json();

  // Pass JSON data directly into your D3.js Sankey renderer
  renderSankeyChart(graphData);
}
```

2. Java Spring Boot → PostgreSQL

- **Protocol:** JDBC via **Spring Data JPA** (Hibernate).
- **How it works:** Spring Boot maps Java classes (`@Entity`) directly to PostgreSQL tables. You define interfaces extending `JpaRepository`, and Spring handles writing and executing SQL queries automatically.
- **Example Java Repository:**

```
@Repository
public interface FinancialNodeRepository extends JpaRepository<FinancialNode, Long> {
  List<FinancialNode> findByUserId(Long userId);
}
```

3. Java Spring Boot → Python Service

- **Protocol:** HTTP REST (Spring `RestClient` / `WebClient` calling Python FastAPI/Flask on `localhost:5000`).
- **How it works:** When a user requests a Monte Carlo simulation or AI advice, Spring Boot collects the user's graph data from PostgreSQL and sends an HTTP POST request to Python. Python processes the math/AI and returns a JSON payload back to Java, which forwards it to the frontend.
- **Example Java Controller Delegate:**

```
@Service
public class SimulationService {
  private final RestClient restClient = RestClient.create("http://python-service:5000");

  public SimulationResult runMonteCarlo(GraphDataPayload payload) {
    return restClient.post()
      .uri("/simulate")
      .contentType(MediaType.APPLICATION_JSON)
      .body(payload)
      .retrieve()
      .body(SimulationResult.class);
  }
}
```

4. Python Service → Claude API & External Market Data

- **Protocol:** HTTPS API calls over the web via requests or standard SDKs.

- **How it works:** Python exposes light endpoints (e.g., `/simulate` or `/optimize-strategy`). It calls `yfinance` to fetch live stock returns and the `anthropic` Python package to query Claude, formatting the output into clean JSON for Spring Boot.
- **Example Python Endpoint (FastAPI):**

```
from fastapi import FastAPI
import yfinance as yf
from anthropic import Anthropic

app = FastAPI()

@app.post("/simulate")
def run_simulation(data: dict):
    # 1. Get market historical return
    ticker = yf.Ticker("V00")
    hist = ticker.history(period="10y")

    # 2. Run simulation math...

    return {"status": "success", "percentiles": [10, 50, 90], "growth_curve": [...]}
```

How To Package and Run Everything Locally (Docker Compose)

To ensure your whole team can run this with one command without configuring local databases or Java/Python paths, place a `docker-compose.yml` file at the root of your GitHub repository:

```
version: '3.8'

services:
  # 1. Database
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: finance_db
      POSTGRES_USER: spring_user
      POSTGRES_PASSWORD: spring_password
    ports:
      - "5432:5432"

  # 2. Python AI & Math Engine
  python-service:
    build: ./python-backend
    ports:
      - "5000:5000"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}

  # 3. Java Spring Boot (Serves REST API + Host Static HTML/JS)
  java-backend:
    build: ./java-backend
    ports:
      - "8080:8080"
    depends_on:
      - postgres-db
      - python-service
```

```
environment:
```

- SPRING_DATASOURCE_URL=jdbc:postgresql://postgres-db:5432/finance_db
- PYTHON_SERVICE_URL=http://python-service:5000

Serving the Frontend

You can place your static `index.html`, `styles.css`, and `app.js` directly inside Spring Boot's `src/main/resources/static/` folder. Spring Boot will automatically serve your web app at `http://localhost:8080` when you open your browser.

User prompt: ok give me an overview on what the project is explained exactly and what it looks to address and accomplish, and how it's going to work and how it will adress a customer or a normal user and why the project is meaningful. Also give a two week structure of tasks and things that each developer needs to do and accomplish, the team will be split the following way: group of two will focus on the python aspect one person will focus on the sql aspect group of two will focus on the java spring one person will focus on the html, css, js

Response: Project Overview & Product Vision

Product Name: Financial Cascade — Visual Cash-Flow Engine & Wealth Simulator

What the Project Addresses & Accomplishes

Traditional personal finance tools rely on static spreadsheets, text budgets, or pie charts that show where money went after it was spent. They fail to help users answer fundamental forward-looking questions: *"What happens to my 10-year net worth if I increase my 401(k) match by 3% while aggressively paying down \$20,000 in student debt?"*

Financial Cascade bridges this gap by modeling personal finance as a dynamic, node-based cash-flow pipeline (similar to a circuit diagram or water grid). Income flows in from the top, passes through customizable rule nodes (taxes, living expenses, high-yield savings, debt payoff, investment accounts), and visualizes compounding net worth trajectories in real time.

How It Works for the End User

1. **Interactive Node Canvas:** Users drag, drop, and connect visual nodes representing income streams, bank accounts, debt balances, and investment portfolios on a drag-and-drop web canvas.
2. **Dynamic Scenario Slider & Simulation Engine:** Users adjust financial levers (salary raises, inflation rates, market return expectations, debt payoff strategies like Avalanche vs. Snowball) and trigger a 1,000-run Monte Carlo simulation based on live S&P 500 and economic metrics.
3. **AI Financial Co-Pilot (Claude Integration):** The user clicks "Optimize Strategy." Claude reads the current graph's state and financial constraints, highlighting inefficient cash leaks, recommending debt restructuring, and presenting a side-by-side visual comparison of their optimized trajectory.

Why This Project Is Meaningful

- **Solves a Real Human Problem:** It turns intimidating, abstract compound interest math into an intuitive visual map, empowering users to make proactive wealth-building decisions.
- **Demonstrates Production-Grade Engineering:** It integrates four distinct tech stacks (Java Spring Boot, Python FastAPI, PostgreSQL, and D3.js/HTML5) into a unified, containerized local microservices architecture using Docker Compose.

Team Role Distribution

- **Python Team (2 Developers):** Focuses on live market API integration (`yfinance`), Monte Carlo simulation math (`NumPy/Pandas`), `FastAPI` endpoints, and Claude LLM prompt pipeline engineering.
- **Java / Spring Boot Team (2 Developers):** Focuses on core REST API design, financial calculation utilities (amortization/tax logic), business rule validation, and orchestration between Python, SQL, and the Frontend.
- **SQL Specialist (1 Developer):** Focuses on relational schema design in PostgreSQL, database indexing, seed archetype data generation, and JPA/Hibernate entity mappings.
- **Frontend Engineer (1 Developer):** Focuses on HTML5/Tailwind CSS styling, responsive dashboard layout, chart visualizers (`Chart.js`), and the `D3.js` interactive cash-flow node canvas.

Sprint-by-Sprint Plan (4 Weeks / Two 2-Week Sprints)

Sprint 1 (Weeks 1 & 2): Foundation, Architecture & Core Visuals

Goal: Establish local Docker environment, build base DB schema, set up REST APIs, and render a basic node canvas.

- **Python Developer 1:**
 - Build `FastAPI` skeleton service with `/health` and `/simulate` endpoints.
 - Write the `yfinance` market data fetcher to pull 10-year historical return averages for index funds (S&P 500, Bond indices).
- **Python Developer 2:**
 - Implement the initial Monte Carlo probability simulation script using `NumPy`.
 - Configure Anthropic Claude API client and draft initial prompt templates for financial graph analysis.
- **SQL Specialist:**
 - Design PostgreSQL relational schema (`users`, `nodes`, `edges`, `financial_profiles`, `simulation_runs`).
 - Create SQL seed scripts for 3 initial user archetypes ("Recent Grad", "Mid-Career Family", "FIRE Strategist").
- **Java Developer 1:**
 - Initialize Spring Boot project structure with JPA entities mapped to PostgreSQL.
 - Create Docker Compose file (`docker-compose.yml`) uniting Spring Boot, Python `FastAPI`, and PostgreSQL.
- **Java Developer 2:**
 - Build REST API endpoints (`/api/v1/profile`, `/api/v1/nodes`) for profile management and CRUD operations.
 - Write Java financial calculation utilities for tax brackets, fixed expenses, and compound interest math.
- **Frontend Engineer:**

- Build responsive HTML5/Tailwind CSS dashboard shell with navigation, slider controls, and panel drawers.
- Implement initial D3.js Sankey or directed graph renderer using static JSON mockup data.

Sprint 2 (Weeks 3 & 4): Integration, AI Features, Polish & Presentation

Goal: Connect all microservices, run end-to-end simulations with Claude AI advice, and prepare demonstration assets.

- **Python Developer 1:**

- Connect Monte Carlo output payloads to return 10th, 50th, and 90th percentile wealth curves to Java.
- Optimize execution speed for probability runs to under 500ms.

- **Python Developer 2:**

- Finalize Claude integration endpoint (`/optimize-strategy`) that parses financial node graphs and outputs structured JSON refactoring tips.
- Implement error handling and rate-limiting fallbacks for API calls.

- **SQL Specialist:**

- Optimize database queries with proper indexes on foreign keys (`user_id`, `node_id`).
- Add transactional support and migration scripts for database snapshot persistence.

- **Java Developer 1:**

- Implement Spring `RestClient` delegates to orchestrate requests between Frontend, PostgreSQL, and Python.
- Add error-handling advice, CORS configuration, and logging for all controller routes.

- **Java Developer 2:**

- Build debt payoff engine comparison logic (Avalanche vs. Snowball method outputs).
- Package static frontend assets (`index.html`, `app.js`, `styles.css`) into Spring Boot static resources for single-port delivery.

- **Frontend Engineer:**

- Wire frontend `fetch()` calls to Spring Boot endpoints for live node manipulation and state updates.
- Integrate Chart.js for multi-scenario net worth line graphs and add slide-over AI recommendation drawer UI.

User prompt: cool, that's a good overview, now do a detailed long list of tasks for each developer let's focus on the first two weeks

Response: **Sprint 1: Foundation, Infrastructure, and Core Visuals (Weeks 1-2)**

Python Developer 1: API Core & Market Data Integration

- Initialize the Python virtual environment and `requirements.txt` (FastAPI, Uvicorn, yfinance, requests).
- Build the base FastAPI application structure and configure a `/health` endpoint for Docker routing checks.
- Integrate the `yfinance` library to fetch 10-year historical pricing data for core tickers (e.g., S&P 500 'VOO', US Bonds 'AGG').
- Write utility functions to calculate the historical Compound Annual Growth Rate (CAGR) and annualized volatility from the fetched ticker data.
- Create a `/api/market-data` endpoint that accepts a ticker symbol and returns the calculated CAGR and volatility in JSON format.
- Write the `Dockerfile` for the Python service to expose port 5000.

Python Developer 2: Math Engine & AI Skeleton

- Add `numpy` and `pandas` to the project dependencies.
- Draft the core Monte Carlo simulation algorithm: a function that takes starting balance, monthly contribution, CAGR, volatility, and time horizon to run 1,000 random market walks.
- Format the Monte Carlo output to return standard arrays for the 10th, 50th, and 90th percentile growth curves.
- Set up the Anthropic Claude Python SDK and configure environment variable loading for the API key (`.env` file).
- Create a placeholder `/api/optimize` endpoint that accepts a JSON graph structure and returns a hardcoded mock JSON response (to unblock Java developers while you refine the actual Claude prompts in Sprint 2).

SQL Specialist: Schema Design & Archetype Seeding

- Install and configure a local PostgreSQL instance (or use a standalone Docker container for testing).
- Draft the Data Definition Language (DDL) SQL scripts for the `users` and `financial_profiles` tables.
- Design the `financial_nodes` table to store distinct types of accounts (columns: `id`, `user_id`, `node_type`, `name`, `current_balance`, `interest_rate_apr`).
- Design the `node_edges` table to represent cash flow routing between nodes (columns: `source_node_id`, `target_node_id`, `percentage_flow`, `fixed_amount_flow`).
- Write a robust `init.sql` seed script containing `INSERT` statements for three complete user archetypes (e.g., "Recent Grad with Loans", "Mid-Career Family").
- Format the seed script to automatically execute when the PostgreSQL Docker container spins up.

Java Developer 1: Architecture, Docker & JPA Mapping

- Initialize the Spring Boot project via Spring Initializr (Web, Spring Data JPA, PostgreSQL Driver, Lombok).
- Create the master `docker-compose.yml` file at the repository root to network the PostgreSQL database, Python FastAPI container, and Spring Boot application together.
- Configure the `application.yml` file to read database credentials and the Python service URL from Docker environment variables.
- Create the JPA `@Entity` classes mapping exactly to the SQL tables (`User`, `FinancialNode`, `NodeEdge`).

- Define the Spring Data JPA Repository interfaces (e.g., `FinancialNodeRepository`) for standard database interactions.

Java Developer 2: REST Controllers & Data Transfer Objects (DTOs)

- Define Data Transfer Objects (DTOs) for the financial graph to ensure raw database entities are not directly exposed to the frontend JSON payload.
- Build the `GraphController` with a GET endpoint (`/api/v1/users/{userId}/graph`) that fetches a user's nodes and edges and packages them into a single graph DTO.
- Implement basic CRUD POST/PUT endpoints allowing the frontend to create a new node (e.g., adding a savings account) or update an existing node's balance.
- Integrate Swagger/OpenAPI (`springdoc-openapi-ui`) to auto-generate API documentation so the frontend developer knows exactly what JSON structure to expect.
- Test all endpoints against the SQL Specialist's seeded database using Postman.

Frontend Engineer (HTML/CSS/JS): UI Layout & D3.js Canvas

- Initialize the frontend project directory and set up Tailwind CSS (or standard CSS modules) for styling.
 - Build the static layout shell: a top navigation bar, a left-hand sidebar for parameter sliders (income, retirement age), and a main central canvas area.
 - Implement dark-mode color variables to ensure the financial data pops visually against the background.
 - Create a hardcoded mock JSON object representing a basic cash-flow graph (Nodes: Salary, Checking, Rent, 401k; Edges connecting them).
 - Integrate the D3.js library and write the initialization code to render the mock JSON data as a network graph or Sankey diagram (drawing rectangles for nodes and curved SVG paths for cash flow).
 - Add basic interactivity to the D3.js canvas: implement hover tooltips that display a node's exact dollar amount when the user's mouse rests on it.
-